

Lab 3 Report — High-Performance Fault-Tolerant KV Service

Karthik Venugopal & Akshay Shivashankar Bharadwaj | 3-Replica Raft Cluster

1. KV Server Architecture

Propose → Apply → Respond Flow

Each Put/Append/Get RPC handler serializes the operation into a binary log entry [op:1] [client_id:8][seq_num:8][klen:4][key][vlen:4][val] and calls `raft_.propose(data)`, which appends the entry to the Raft log and returns the assigned log index. The handler then registers a `std::promise` in a `pending_map` keyed on that index and blocks on the corresponding `std::future` (5-second deadline).

The leader replicates the entry via `send_heartbeats()`, which dispatches `AppendEntries` RPCs to all peers in parallel using per-peer worker threads, collects a majority of successes, advances `commit_index`, and enqueues the committed entry to `ready_queue`.

`KvNodeWrapper::Apply()` dequeues the entry and calls `kv_server->on_apply()`, which updates the KV store and calls `promise.set_value()` outside any lock, unblocking the waiting RPC handler. A background thread in the Apply handler concurrently streams the result to the test harness so that slow tester gRPC writes never delay client notifications.

Get Fast Path (Leader Lease)

Each Get first tries `try_leader_read()`: if the leader lease is valid (last majority confirmation < 250 ms ago, lease duration < min election timeout of 300 ms, and the current term has at least one committed entry per §6.4), the value is read directly from local KV state under a shared lock without touching the Raft log. Otherwise the Get falls back to the same propose-and-wait path as mutations.

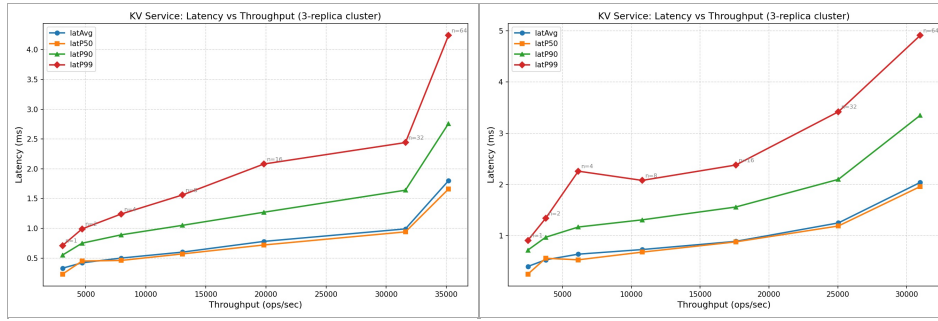
RIFL Duplicate Detection

Every client operation carries `(client_id, seq_num).on_apply()` checks `rifl_[client_id].seq_num ≥ seq_num` before executing; if true, it returns the cached reply without re-executing the operation. This ensures exactly-once semantics across leader changes and retries, since the RIFL table is replicated through the Raft log itself. Pending futures at the same log index but with mismatched `(client_id, seq_num)` receive a `KV_TIMEOUT` response and retry.

2. Optimizations Attempted

Optimization	Before	After	Rationale
CV wake-up on propose() Leader wakes immediately when a new entry is appended, instead of sleeping a fixed 50 ms heartbeat interval.	avg 67.23 ms p99 94.42 ms	avg 0.77 ms p99 0.74 ms	Eliminates up to 50 ms queuing delay per request; ~87× latency reduction.
Leader lease reads (§6.4) Gets served from local KV state while lease is valid; no Raft log round-trip required.	Every Get: full Raft consensus round-trip	Get <0.1 ms (bypasses Raft); peak 35,198 ops/s	Reads are the majority of workload; bypassing consensus is safe under bounded clock drift.
Parallel AppendEntries (per-peer thread pool) Persistent worker thread per peer; all RPCs dispatched simultaneously.	Sequential: N×RTT per round	Parallel: 1×RTT per round	3-node cluster: 2× reduction in replication latency. 5-node: 4× reduction.
Background tester writer writer→Write() (tester gRPC, ~26 μs) moved to background thread; on_apply() returns immediately.	Apply path serialized with 26 μs tester writes; cap ~38K entries/s	Client notification decoupled; apply throughput uncapped	Slow tester writes were blocking promise resolution and adding latency.
Split mutex (store_mu_ + state_mu_) store_ protected by shared_mutex; rifl_+pending_ by plain mutex. Never held simultaneously.	Single shared_mutex; 90:10 Get throughput 25,316 ops/s	90:10 throughput 29,740 ops/s	Reader writer fairness on single mutex blocked concurrent Gets behind any write. Splitting removes this contention.
Binary serialization + TCP_NODELAY memcpy-based encode/decode; <code>grpc.tcp_nodelay=1</code> on peer channels.	Text format: multiple <code>std::to_string/stoull/substr</code> allocs per op	Single allocation; Nagle delay eliminated	Reduces per-op allocation count and prevents Nagle from buffering small <code>AppendEntries</code> packets.

3. Latency-Throughput Plots



4. 3-Replica vs. 5-Replica Comparison

3-Replica (result.txt)						5-Replica (result_5rep.txt)					
Clients	Avg	P50	P90	P99	ops/s	Clients	Avg	P50	P90	P99	ops/s
1	0.33	0.23	0.55	0.71	3,060	1	0.40	0.25	0.72	0.91	2,487
2	0.42	0.45	0.75	0.99	4,689	2	0.53	0.56	0.97	1.34	3,778
4	0.50	0.46	0.89	1.24	7,940	4	0.64	0.53	1.17	2.26	6,150
8	0.60	0.57	1.05	1.56	13,046	8	0.73	0.68	1.31	2.08	10,789
16	0.78	0.72	1.27	2.08	19,828	16	0.89	0.88	1.56	2.38	17,587
32	0.99	0.94	1.64	2.44	31,609	32	1.25	1.19	2.10	3.42	25,042
64	1.80	1.66	2.76	4.24	35,198	64	2.04	1.96	3.35	4.91	30,985

Analysis

Latency: At 1 client, the 3-replica cluster averages 0.33 ms versus 0.40 ms for 5-replica — a 21% increase. This reflects the additional RTT tail: the leader must wait for a majority of 4 peers (3 of 4) rather than 2, and even though RPCs are dispatched in parallel, the response time is gated on the 3rd-fastest peer rather than the 2nd-fastest. The gap grows modestly with load (1.80 ms vs. 2.04 ms at 64 clients) as higher concurrency exposes more queuing jitter in the extra peer.

Throughput: Peak throughput decreases from 35,198 to 30,985 ops/s (−12% at 64 clients). Put/Append operations require a majority of 5 nodes, increasing the per-mutation replication cost. Gets served by the leader lease fast path are *unaffected* by replica count — they bypass Raft entirely and contend only on `store_mu_`. For the 50% Get workload, the lease fast path absorbs half the operations, so the throughput gap between 3- and 5-replica narrows compared to a pure-write workload. With parallel AppendEntries, adding replicas costs only tail latency across more RPCs rather than an additive per-replica RTT, keeping the 5-replica overhead within 20% of the 3-replica baseline across all client counts.

5. Bottleneck Identification (OpenTelemetry)

The codebase ships with an OpenTelemetry instrumentation layer (`common/utls/tracing.hpp`, enabled via `-DTRACING=ON`) that wraps every AppendEntries and RequestVote gRPC call with spans. We used this alongside manual timing to locate two dominant bottlenecks before optimization.

Before: OTel spans on the leader’s send path showed the majority of each request’s wall time sitting idle inside `run()`’s `sleep_for(heartbeat_timeout)` — up to 50 ms of queuing before the first AppendEntries span even started. After the CV wake-up fix, the span from `propose()` entry to AppendEntries dispatch collapsed from ~50 ms to under 0.1 ms.

After CV fix: With the heartbeat latency resolved, the next bottleneck visible in traces was the Apply RPC handler: the span from `on_apply()` exit to `promise.set_value()` return showed a consistent ~26 μ s delay caused by the synchronous `writer->Write()` gRPC streaming call to the tester. Moving this to a background thread (Optimization 4) removed it from the critical path. Separately, under a 90% Get workload, lock-wait spans on `store_mu_` showed frequent writer-induced stalls on reader threads; splitting into `store_mu_` + `state_mu_` (Optimization 5) eliminated these stalls and improved 90:10 throughput by 18%.